LSTM Based Internet Traffic Forecasting for Intelligent Bandwidth Prediction

Sahasrad S Nair-24BDS0405, Shlok Malik-24BDS0280
Department of Computer Science and Engineering, VIT Vellore, Tamil Nadu, India

Abstract—Traffic forecasts can help plan bandwidth, but lower prediction error does not always mean fewer shortages or less unused capacity. We study this relationship through an LSTM-based prototype using institution-level traffic from CESNET-TimeSeries24. Persistence, moving average, LSTM, GRU, and Transformer models predict demand 10, 30, and 60 minutes ahead from three hours of history. Transformer achieves the lowest 10-minute test RMSE at 30.803 Mbps, while GRU leads at 30 and 60 minutes with 32.290 and 34.785 Mbps. A separate 10-minute allocation replay compares all five policies. At 20% headroom, LSTM reduces shortfall intervals from 23.443% for reactive allocation to 10.857%, while increasing mean capacity from 140.214 to 148.669 Mbps. No evaluated policy meets the 1% validation shortfall target. The prototype includes a dashboard, saved predictions, and a verification workflow; all 41 program tests pass. The results show why forecasting and allocation should be assessed together. They describe one institution and one training seed per configuration, with broader evaluation needed before deployment.

Keywords—Network traffic forecasting, intelligent bandwidth prediction, LSTM, GRU, Transformer, proactive allocation, reproducible evaluation.

1 INTRODUCTION

Bandwidth planning involves a practical tradeoff. Too little capacity leaves demand unserved in the allocation model, while too much leaves resources idle. A forecast can help operators plan ahead, but prediction accuracy alone does not tell them how much capacity to reserve. The direction of the error, the level of demand, and the safety margin all influence that decision.

We examine this tradeoff using traffic from CESNET-TimeSeries24 [1, 2]. The prototype combines data preparation, forecasting, capacity replay, and a dashboard. LSTM is the project's primary model; GRU and Transformer provide architectural comparisons, while persistence and moving average provide simple baselines. Running them under the same protocol lets us compare both prediction errors and the capacity decisions that follow.

We ask three questions: Do learned models improve on simple predictors when evaluated on the same observations? How does forecasting further ahead change their accuracy and ranking? How much capacity is needed to reduce shortages? The first two questions concern forecasting; the third requires a separate allocation experiment.

Our contribution is a reproducible case study that connects these questions in one working program. It includes a causal preprocessing policy, 15 forecasting experiments across three horizons, and a five-policy allocation comparison at 10 minutes. The saved predictions allow each reported metric to be checked against the underlying records.

The experiments concern one institution and an offline allocation model. They do not measure queueing delay, packet loss, routing changes, or controller response time. We therefore interpret the results as evidence for bandwidth planning, with live deployment left for further study.

2 RELATED WORK

LSTM uses gated memory to learn dependencies in sequences [3], while GRU uses a different recurrent gating structure [4]. Transformers represent sequences through attention [5]. These approaches offer useful alternatives for traffic forecasting, although their original benchmarks cannot determine which works best here. In our comparison, every neural model receives the same three hours of history.

Azzouni and Pujolle's NeuTM uses LSTM for traffic-matrix prediction on GÉANT measurements [6]. It provides a relevant precedent for learning traffic patterns, but its task differs from predicting the aggregate rate of one institution. We use it to motivate the model choice rather than as a numerical benchmark for the CESNET results.

DeepCog predicts capacity with a loss that accounts for both underprovisioning and overprovisioning in network slicing [7]. Tuna et al. compare mobile-traffic predictors and parametric losses under service-level constraints [8]. Both studies motivate evaluating the allocation decision directly. Here, we train predictors with squared error and select a safety margin afterward. This makes the effect of the margin easy to inspect, although it does not optimize shortages during training.

Zeng et al. found simple linear methods competitive with more elaborate models on their long-term forecasting tasks [9]. Their results reinforce the need for useful baselines. Persistence and moving average test whether our learned models improve on continuation and local smoothing; seasonal and linear predictors would make that comparison more complete.

CESNET-TimeSeries24 contains time series from an operational academic network [1, 2]. Its documented provenance supports repeatable experiments, but interruptions in measurement still require care. We explicitly track excluded intervals and align forecasts by timestamp. Because prior studies use different targets, horizons, and resource costs, we compare their methods rather than rank their published errors against ours.

3 DATA AND FORECASTING TASK

3.1 Traffic representation and quality control

We use institution 0, which has 40,298 mapped observations. An availability check across 283 institution files found 56 with every supplied timestamp identifier; selection was based on coverage rather than forecast performance. Joining observations to the supplied time mapping reveals five irregular transitions, which divide the series into six continuous segments.

We exclude May 20 through June 5, 2024, inclusive, using the supplied mapping's calendar dates, and guard the first

and last record of each continuous segment. The outage accounts for 2,448 rows and the boundary guards for 12, leaving 37,838 eligible observations before forecasting windows are formed. Windows cannot cross an exclusion or segment boundary. Raw records remain unchanged, so excluded time is never compressed into an apparently continuous history.

If $B(t)$ is the recorded byte volume for a nominal 600-second interval, its target rate is

$$y(t) = \frac{8B(t)}{600 \times 10^6} \quad \text{nominal Mbps.} \tag{1}$$

This conversion uses decimal megabits and the nominal interval length. It describes the recorded rate, not physical-link utilization. We also avoid summing institution records into a network-wide demand estimate, since endpoint accounting could count the same traffic more than once.

The exclusion policy defines where the reported accuracy applies. It says nothing about performance during the outage. Observed zeros remain distinct from missing measurements, and any future change to the exclusions should be evaluated on shared targets as well as the newly eligible set.

3.2 Direct targets and causal features

Let t denote the last completed historical interval. An input $\mathbf{X}_t$ contains 18 historical observations and eight features, and a separate model is fitted for each lead:

$$\hat{y}_{t+k} = f_{\theta,k}(\mathbf{X}_t), \qquad \mathbf{X}_t \in \mathbb{R}^{18\times 8}, \quad k \in \{1,3,6\}. \tag{2}$$

A 30-minute forecast predicts the rate in the single 10-minute interval ending three steps after the origin. It does not predict total demand across those three steps. The same convention applies at 60 minutes, so deficit volume is always calculated for one target interval.

Each observation has eight features: traffic, sine and cosine encodings of hour of day and day of week, a missing-input indicator, and a rolling mean and population standard deviation. Rolling statistics use up to six available positions within the input window. Historical filling uses only earlier observations and resets at exclusions; target values must be observed.

We fit scalers on training windows separately for each horizon. With target mean μ_y and scale σ_y, the standardized target is $z = (y - \mu_y)/\sigma_y$ and the forecast is restored as $\hat{y} = \mu_y + \sigma_y \hat{z}$. A zero standard deviation is replaced by one. Validation and test data never update the scalers. The input scaler pools overlapping training-window positions, consistently across models.

We assume that an input interval is available when it ends. No future target enters the feature tensor. A deployed system would also need to account for collection and processing delays, which reduce the time available to act on a forecast.

4 FORECASTING AND CAPACITY METHODOLOGY

4.1 Model comparison

Persistence repeats the latest available rate. Moving average uses the four latest rates, $\hat{y}_{t+k} = \frac{1}{4}\sum_{j=0}^{3} y_{t-j}$, with the same causal handling of inputs as the neural models. Neither baseline has learned neural parameters. They establish whether sequence learning improves on a recent observation or a smoothed history.

LSTM and GRU both have two recurrent layers, 64 hidden units, zero dropout, and a linear output from the last historical position. Their parameter counts are 52,289 and 39,233. Hidden state resets for each window. For LSTM, let u_j concatenate the layer input and previous hidden state:

$$\begin{aligned} i_j &= \sigma(W_i u_j + b_i), & f_j &= \sigma(W_f u_j + b_f), \\ g_j &= \tanh(W_g u_j + b_g), & o_j &= \sigma(W_o u_j + b_o), \\ c_j &= f_j \odot c_{j-1} + i_j \odot g_j, \\ h_j &= o_j \odot \tanh(c_j). \end{aligned} \tag{3}$$

The gates control which information is retained, added, and passed onward [3]. The second layer receives the first layer's hidden sequence and predicts one standardized target. Since state resets between windows, the model's effective history is the supplied three-hour window.

The Transformer maps the eight features to 64 dimensions and adds fixed sinusoidal position encodings. Its two encoder layers have four attention heads, a feedforward width of 128, GELU activation, and zero dropout. A scalar output reads the final historical position. The model has 67,585 parameters. All attention inputs belong to known history, so an unrestricted mask within that history does not reveal future targets.

4.2 Capacity rule and validation selection

For predictor m and margin α, the simulated capacity is

$$C_i(m,\alpha) = (1+\alpha)\max(0, \hat{y}_i^{(m)}). \tag{4}$$

Reactive allocation uses the last observed demand in place of the forecast. We leave predictions unclipped when measuring forecast error and apply the nonnegative constraint only when allocating capacity. The replay covers five policies at margins of 5%, 10%, 15%, and 20%, producing 20 settings per split and 40 validation/test summaries. Allocation is currently evaluated only at 10 minutes.

For n scored intervals, the principal resource metrics are

$$\begin{aligned} S &= \frac{100}{n}\sum_i \mathbf{1}[y_i > C_i], & \bar{C} &= \frac{1}{n}\sum_i C_i, \\ U &= \frac{1}{n}\sum_i \max(C_i - y_i, 0), \\ D &= \frac{600}{8}\sum_i \max(y_i - C_i, 0). \end{aligned} \tag{5}$$

Here S is the percentage of intervals with a shortfall, $\bar{C}$ is mean capacity, and U is mean excess capacity, both in Mbps. Deficit volume D is measured in decimal MB; excluded intervals contribute no volume. Frequency and volume capture different aspects of a shortage, since equally frequent failures can leave very different amounts of demand unserved.

For each policy, we choose the margin with the lowest mean validation capacity among those satisfying $S_{\text{val}} \le 1\%$. If none qualifies, the program uses a documented 20% fallback. Test data do not influence this choice. Other points in the margin grid help explain the tradeoff, but remain exploratory comparisons.

4.3 Why forecast error and allocation quality differ

With positive demand and a nonnegative forecast, let $e_i = \hat{y}_i - y_i$. Equation (4) implies

$$y_i > C_i \quad \Longleftrightarrow \quad e_i < -\frac{\alpha}{1+\alpha} y_i. \tag{6}$$

At 20% headroom, a shortage occurs when underprediction exceeds one sixth of actual demand. Shortages therefore depend on the negative tail of the errors relative to demand, which MAE and RMSE do not fully describe. A predictor that tends to overestimate demand can reduce shortages while reserving more capacity.

A loss such as $c_u \max(y - C, 0) + c_o \max(C - y, 0)$ could assign different costs to shortages and unused capacity. Those costs are unknown for this institution, so we do not infer monetary savings. Cost-aware losses and calibrated upper-demand estimates are useful extensions suggested by earlier resource-allocation work [7, 8].

5 EXPERIMENTAL PROTOCOL AND SYSTEM

5.1 Temporal evaluation and optimization

We split the mapped timeline chronologically into 70% training, 15% validation, and 15% test before applying exclusions. Target completion determines membership, while earlier observations can supply history to a later split. The nominal cuts are April 21, 2024, at 22:40:52 UTC and June 2 at 22:10:52 UTC. The test cut lies within the outage, so eligible test observations begin after the exclusion.

Table 1 Prepared chronological window counts

Lead min	Training	Validation	Test
10	28,129	4,029	5,554
30	28,121	4,029	5,548
60	28,109	4,029	5,539

Table 1 gives the usable window counts after filtering. Longer leads remove some targets near boundaries. All models at a given horizon share the same targets; we also compare the 5,539 targets shared by all three horizons to check whether coverage differences explain the horizon effect.

We train with seed 42, Adam at learning rate 0.001, batch size 64, and mean squared error on standardized targets. Training stops after at most 30 epochs or five epochs without validation improvement. A seeded generator shuffles training windows; validation and test remain chronological. Predictions come from the checkpoint with the lowest validation loss.

These settings give a repeatable starting comparison, rather than an exhaustive search for each architecture. Further tuning needs a separate development protocol. Since the present test results are already known, confirmatory experiments should use an untouched period or institution.

5.2 Metrics and reproducibility

For residuals $e_i = \hat{y}_i - y_i$, prediction errors are computed in original units:

$$\mathrm{MAE} = \frac{1}{n}\sum_i |e_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i e_i^2}. \tag{7}$$

We also report MAPE over positive targets and R^2 relative to the evaluation-set mean. Zeros remain in MAE and RMSE. MAPE is unavailable without positive targets, as is R^2 for a constant target series. MAPE gives more weight to relative errors at low demand, whereas RMSE emphasizes large absolute errors. Their rankings can therefore differ for understandable reasons.

We check the reported values by recomputing metrics from saved predictions, matching target identifiers, and comparing prepared-data hashes with run configurations. The full allocation grid is recomputed from its forecasts. Saved configurations, timestamps, scalers, and checkpoints allow the calculations to be traced back to the source records.

5.3 Implemented system and evaluation coverage

Figure 1 shows the program's data flow. A Streamlit dashboard reads saved experiments for all three forecasting horizons and compares the five allocation policies at 10 minutes. Users can select a model, horizon, and split without retraining. Plots retain actual timestamps and excluded gaps, while CSV exports preserve units and experiment provenance.

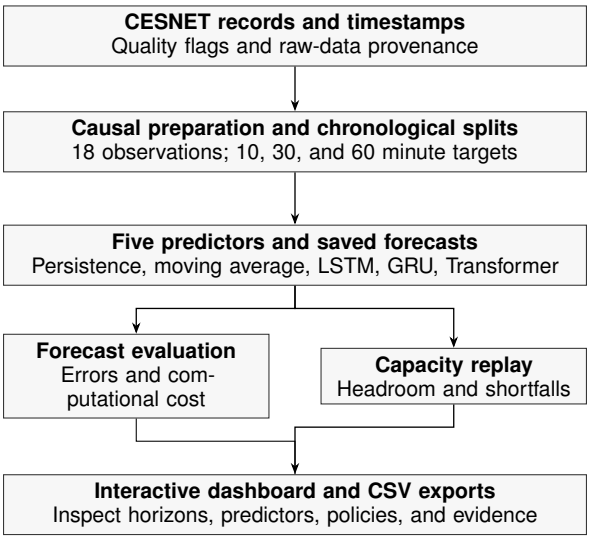


Figure 1 Implemented forecasting and allocation framework. Forecasting covers 10, 30, and 60 minutes; the five-policy allocation replay covers 10 minutes.

The dashboard distinguishes the 20% fallback from exploratory margin choices. Replay controls change the visible interval history, while headline metrics summarize the full eligible split. Allocation at 30 and 60 minutes is marked unavailable. The application supports inspection of the experiments; it does not control a live network.

6 MEASURED RESULTS AND DISCUSSION

6.1 Prediction accuracy and model ranking

Table 2 Measured 10 minute test results. MAE and RMSE are nominal Mbps; MAPE is percent.

Model	MAE	RMSE	MAPE	R^2
Persistence	24.287	38.629	22.933	0.570
Moving average	21.025	32.478	20.608	0.696
LSTM	21.148	31.175	21.191	0.720
GRU	20.910	31.006	20.921	0.723
Transformer	20.746	30.803	21.864	0.726

Tables 2–4 report the test metrics, and Figure 2 compares the three error measures across horizons. At 10 minutes, Transformer has the lowest MAE and RMSE, while moving average has the lowest MAPE. Transformer's RMSE is 20.26% below persistence but only 1.20% below LSTM. The improve-

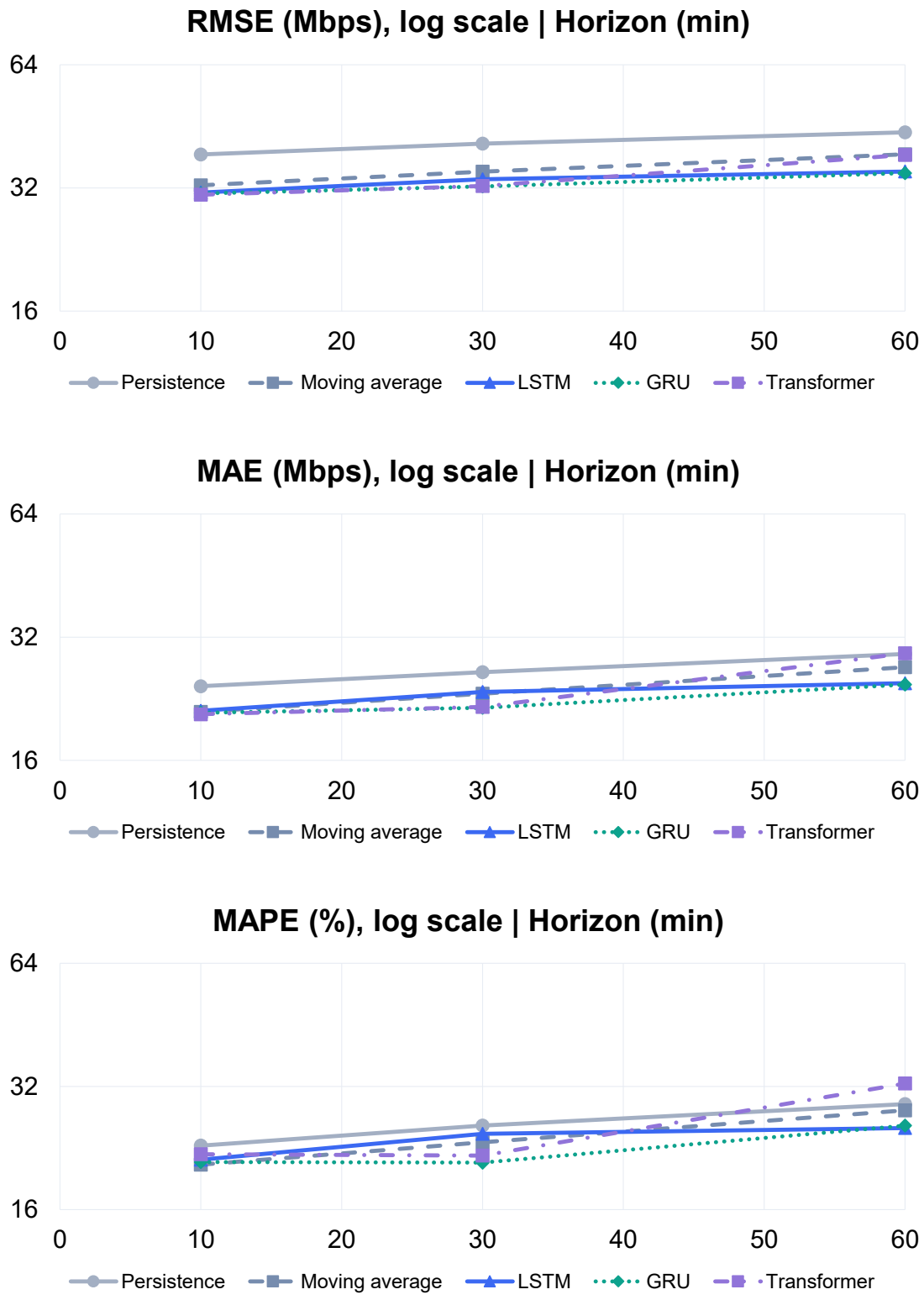


Figure 2 Test errors across forecast horizons. The vertical axes are logarithmic (base 2): equal distances represent equal ratios, not equal absolute differences. Markers show measured 10-, 30-, and 60-minute results; connecting lines guide the eye and do not represent additional experiments. Each horizon uses the targets in Table 1. RMSE and MAE are in Mbps; MAPE is in percent. Exact values appear in Tables 2–4.

ment over persistence is much clearer than the separation among the neural models.

At 30 minutes, GRU leads on MAE, RMSE, and MAPE, although its RMSE is only 0.051 Mbps below Transformer's.

Table 3 Measured 30 minute test results. MAE and RMSE are nominal Mbps; MAPE is percent.

Model	MAE	RMSE	MAPE	R^2
Persistence	26.270	41.081	25.693	0.513
Moving average	23.294	35.060	23.363	0.646
LSTM	23.502	33.611	24.517	0.674
GRU	21.495	32.290	20.847	0.699
Transformer	21.606	32.340	21.685	0.698

Table 4 Measured 60 minute test results. MAE and RMSE are nominal Mbps; MAPE is percent.

Model	MAE	RMSE	MAPE	R^2
Persistence	29.115	43.754	28.976	0.448
Moving average	27.025	38.668	27.973	0.569
LSTM	24.704	35.091	25.321	0.645
GRU	24.506	34.785	25.666	0.651
Transformer	29.235	38.537	32.530	0.572

LSTM reduces RMSE by 4.14% relative to moving average, but has worse MAE and MAPE. This suggests a different balance of large and ordinary errors. Identifying which demand levels drive that difference requires examining the residuals.

At 60 minutes, GRU again has the lowest MAE and RMSE, while LSTM has the lowest MAPE. GRU's RMSE of 34.785 Mbps improves on persistence by 20.50% and moving average by 10.04%. LSTM follows at 35.091 Mbps. Transformer reaches 38.537 Mbps, close to moving average, and its MAE is worse than both baselines. Its advantage at 10 minutes does not carry through to the longest horizon.

The evaluation period also changes the ranking. GRU has the lowest neural validation RMSE at 10 and 60 minutes, while Transformer leads at 30 minutes. Validation and test winners therefore differ at two horizons. These results describe one seed per configuration; they do not establish statistical significance or justify choosing a deployment model from test performance.

Restricting the comparison to the 5,539 shared test targets preserves the horizon effect: every model's RMSE rises with lead time. From 10 to 60 minutes it rises from 38.644 to 43.754 Mbps for persistence, 32.502 to 38.668 for moving average, 31.190 to 35.091 for LSTM, 31.020 to 34.785 for GRU, and 30.816 to 38.537 for Transformer. The pattern is therefore not explained by the small differences in target coverage.

The forecast comparison also reveals where a simple baseline remains useful. At 10 minutes, moving average has lower MAE than LSTM (21.025 versus 21.148 Mbps) and lower MAPE (20.608% versus 21.191%), even though LSTM has lower RMSE. Squaring errors gives large deviations greater influence, so LSTM's RMSE advantage is compatible with worse performance on the other summaries. It would be premature to attribute the difference specifically to traffic peaks: that claim needs errors grouped by actual demand and examined over time.

The longer-horizon results make the distinction clearer. At 60 minutes, LSTM's MAPE is 25.321%, below GRU's 25.666%, while GRU has lower MAE and RMSE. A planning task concerned with absolute demand errors would interpret this differently from one concerned with proportional errors at low demand. Figure 2 therefore retains all three measures rather than collapsing performance into one score. Its logarithmic axes show proportional changes; the tables retain absolute values for direct comparisons. The separate models fitted at each horizon also mean that the lines describe measured performance across tasks, not the evolution of one recursive forecast.

6.2 Allocation performance and capacity cost

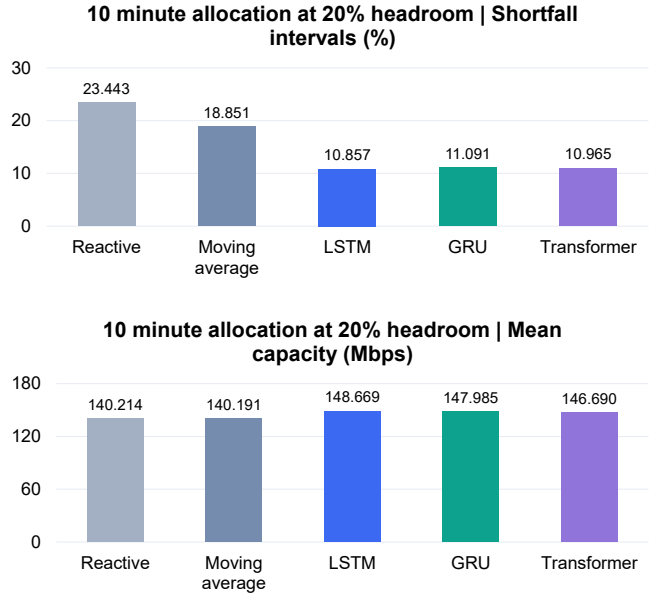


Figure 3 Test allocation at 10 minutes with 20% headroom. Shortfall frequency (top) must be read alongside mean capacity (bottom): fewer shortfalls can require more capacity. All policies use the same 5,554 test intervals.

Table 5 Measured 10 minute allocation at 20 percent margin

Policy	Shortfall %	Capacity Mbps	Excess Mbps
Reactive	23.443	140.214	28.478
Moving average	18.851	140.191	27.394
LSTM	10.857	148.669	34.472
GRU	11.091	147.985	33.886
Transformer	10.965	146.690	32.598

At 20% headroom, LSTM lowers shortfall frequency by 12.59 percentage points relative to reactive allocation, but increases average capacity by 6.03% (Table 5 and Figure 3). Mean excess rises from 28.478 to 34.472 Mbps, while deficit volume falls from about 2.127 million to 1.102 million MB. LSTM serves more demand in the replay, at the cost of reserving more capacity and leaving more unused.

Forecast bias helps explain this result. Mean signed error at 10 minutes is approximately 0.002 Mbps for persistence, -0.017 for moving average, and $+7.048$ for LSTM. With non-negative forecasts and a fixed margin, mean capacity follows mean predicted demand. LSTM's upward bias therefore accounts for its higher average allocation, although a controlled comparison would be needed to separate the effects of bias and error timing on shortages.

The margin grid also contains an interesting exploratory result. At 10% headroom, LSTM has 18.797% test shortfalls and mean capacity of 136.280 Mbps, improving both quantities over reactive allocation at 20% (23.443% and 140.214

Mbps). However, that LSTM setting has 23.480% validation shortfalls. It illustrates a useful tradeoff without meeting the 1% target.

At 20% headroom, GRU and Transformer have test shortfall rates of 11.091% and 10.965%, using 147.985 and 146.690 Mbps on average. LSTM has the fewest shortfall intervals but reserves the most capacity among the proactive policies. Transformer offers slightly more shortfalls with lower average capacity. The model with the lowest RMSE is therefore not automatically the best allocation policy.

Validation shortfalls at 20% are 19.682%, 16.307%, 13.452%, 13.899%, and 11.963% for reactive, moving-average, LSTM, GRU, and Transformer policies. No candidate satisfies the 1% constraint, so all defaults are fallbacks. Transformer leads on validation while LSTM leads on test. Any revised margin or training objective needs a new validation comparison and an explicit capacity budget.

The allocation records allow the shortage result to be expressed as event counts as well as percentages. Across 5,554 test intervals, reactive allocation has 1,302 shortfalls and LSTM has 603. Transformer has 609 and GRU has 616. Thus, the apparent LSTM advantage over Transformer amounts to only six intervals in this split, whereas its difference from reactive allocation is 699 intervals. The former is a much weaker basis for choosing an architecture, especially because adjacent intervals may share the same traffic event. These counts do not establish that LSTM is more reliable on unseen periods.

Capacity averages also need a clear operational interpretation. The replay permits a different capacity value for each target interval without modelling a physical ceiling, adjustment delay, or cost of changing allocations. Its mean excess is unused capacity within that model; it is not measured idle link capacity or a monetary saving. The failed validation constraint is therefore a useful engineering finding: under the tested margin grid, none of these predictors provides the requested shortage frequency. An operator would need to revise the capacity budget, prediction objective, or service target before treating a selected setting as acceptable.

6.3 Computational cost

Training used an RTX 4050 Laptop GPU. The LSTM, GRU, and Transformer runs took 90.352, 72.262, and 145.143 seconds at 10 minutes; 24.592, 20.132, and 35.726 seconds at 30 minutes; and 44.261, 56.063, and 130.026 seconds at 60 minutes. The nine loops total about 10.31 minutes. These timings include validation and checkpoint saving but exclude initial loading and final evaluation. Early stopping produced different epoch counts, so duration cannot be inferred from horizon alone.

For the 30-minute models, recorded CPU inference times are about 0.814 ms for LSTM, 0.795 ms for GRU, and 0.707 ms for Transformer per preloaded window. Measurements use one CPU thread, 25 warm-up calls, and 1,000 timed calls, including inverse scaling. They exclude data collection and feature construction, so they describe model execution rather than end-to-end service latency.

7 PROGRAM VERIFICATION AND REPRODUCIBILITY

7.1 Executable workflow

The program keeps preparation, forecasting, allocation, and display separate. Each model–horizon directory stores predictions, metrics, configuration, and, for neural models, the checkpoint and training history. Allocation joins predictions by target timestamp rather than row order. The dashboard checks these files before displaying them.

The `scripts.reproduce` entry point runs preparation, all 15 forecasts, the five-policy allocation replay, plots, verification, and tests. Complete runs can be reused after verification; partial outputs are rejected. Fresh runs archive existing generated data and results before recomputation, preserving earlier experiments for comparison.

The main checks are `python -m scripts.verify_results -checkpoints -device auto` and `python -m unittest discover -s tests -v`. Strict checkpoint comparisons should use the saved training backend because CPU and CUDA arithmetic can differ. The dashboard itself needs only the saved results.

7.2 Verification against the saved experiment

Timestamp alignment is particularly important when connecting forecasts to allocation. A forecast is useful only if it refers to the same target interval as the observed demand used to score it. Missing windows can shift row positions without changing the number of columns in a file, so a positional join could produce plausible but incorrect results. The program instead uses target identifiers and retains the forecast origin, allowing the verifier to check both which interval is predicted and whether its inputs were available beforehand. This check links the statistical experiment to the program's actual decision sequence.

The verifier reconstructs windows and training-only scalers, checks data hashes and target alignment, and recomputes forecast metrics and baseline predictions. It also checks checkpoint selection against validation history and replays all nine neural models. Allocation checks recompute capacity, deficits, excess, and selected margins; a hash manifest identifies the files used.

Verification passes at all three horizons, covering 30 forecast metric rows across validation and test. The allocation audit covers 40 metric rows, five default test rows, and 191,660 replay rows. All 41 tests pass, including causal preprocessing, checkpoint restoration, forecast alignment, corrupt-file rejection, and dashboard controls. These checks support confidence in the implementation, while statistical generalization remains a separate question.

A reproduction on October 22, 2026 used a separate clean project copy with the same raw sample and installed Python environment. Accuracy values, all nine checkpoint tensor sets, and allocation records matched. Runtime varied and was excluded from the equality check. This demonstrates repeatability on the workstation, rather than a fresh installation or cross-platform replication.

7.3 Coverage of the completed experiments

The completed experiments show why longer horizons need their own evaluation. The strongest short-horizon model does not remain strongest at 60 minutes, where GRU and LSTM perform better on RMSE. All tables and charts use

measured outputs from the saved runs.

Extending allocation to 30 and 60 minutes requires matching those forecasts to their target intervals, selecting margins on validation data, and reporting coverage separately. Having forecasts at those horizons does not by itself establish their allocation performance.

8 LIMITATIONS AND FUTURE WORK

The main limitation is that we study one institution selected for timestamp coverage. Other institutions may have different demand patterns or poorer measurement quality. A broader evaluation should choose institutions in advance and avoid tuning to their test results. Live deployment would additionally require physical link limits, collection delays, queueing behaviour, and a defined control action.

Overlapping input windows and persistent traffic patterns make neighbouring predictions dependent. Thousands of forecasts are therefore not thousands of independent architecture comparisons. Repeated seeds would measure training variability, while evaluation across separate time blocks would test sensitivity to traffic conditions. Until these experiments are done, small differences between models should be treated cautiously.

Excluding outages and guarding segment boundaries improves the consistency of the input, but changes the population being evaluated. Future sensitivity checks should vary these rules, preserve observed zeros, and compare shared targets. Missing targets must never be replaced by predictions and treated as ground truth. Changing the history length also changes eligibility and needs the same care.

The baseline set and architecture search are limited. Seasonal persistence and a linear forecaster would help establish whether nonlinear recurrence is needed. Removing calendar or rolling features in separate experiments would show how much they contribute. Such comparisons should keep target coverage and validation rules fixed.

Equal headroom does not mean equal capacity expenditure. A useful next experiment would compare shortage rates at matched capacity budgets, then choose operating points on validation data. Calibrated upper-demand forecasts or asymmetric losses could address shortages during training. The failure to meet the 1% target makes these extensions more relevant than simply adding a more complex predictor.

The verification workflow supports the recorded experiments but cannot establish robustness to new traffic. Independent replication should first reproduce target alignment and metrics in original units, then examine whether the findings hold across seeds, periods, and institutions.

9 CONCLUSION

We built and evaluated a forecasting and bandwidth-planning prototype using one institution's traffic. Transformer achieves the lowest 10-minute RMSE, while GRU leads at 30 and 60 minutes. In the 10-minute allocation replay, LSTM has the fewest test shortfalls at 20% headroom, but uses more average capacity than the other proactive policies. No policy meets the 1% validation target.

The practical lesson is to assess forecast accuracy alongside bias, shortage frequency, and capacity use. The dashboard and verification workflow make these relationships inspectable in the saved experiments. Broader baselines, repeated runs, longer-horizon allocation, and operational testing are needed before recommending a model for live network control.

REFERENCES

[1] J. Koumar, K. Hynek, T. Čejka, and P. Šiška, "CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detection and Forecasting," arXiv:2409.18874, 2024. https://arxiv.org/abs/2409.18874.

[2] J. Koumar et al., "CESNET-TimeSeries24: Time Series Dataset for Network Traffic Anomaly Detection and Forecasting," Zenodo dataset, 2024. https://zenodo.org/records/13382427.

[3] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. doi:10.1162/neco.1997.9.8.1735.

[4] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," arXiv:1406.1078, 2014. https://arxiv.org/abs/1406.1078.

[5] A. Vaswani et al., "Attention Is All You Need," arXiv:1706.03762, 2017. https://arxiv.org/abs/1706.03762.

[6] A. Azzouni and G. Pujolle, "NeuTM: A Neural Network-based Framework for Traffic Matrix Prediction in SDN," arXiv:1710.06799, 2017. https://arxiv.org/abs/1710.06799.

[7] D. Bega, M. Gramaglia, M. Fiore, A. Banchs, and X. Costa-Perez, "DeepCog: Optimizing Resource Provisioning in Network Slicing With AI-Based Capacity Forecasting," *IEEE Journal on Selected Areas in Communications*, vol. 38, 2020. doi:10.1109/JSAC.2019.2959245.

[8] E. Tuna, A. Baykal, and A. Soysal, "Multivariate and multi-step mobile traffic prediction with SLA constraints: A comparative study," *Ad Hoc Networks*, vol. 163, art. 103594, 2024. doi:10.1016/j.adhoc.2024.103594.

[9] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers Effective for Time Series Forecasting?" arXiv:2205.13504, 2022. https://arxiv.org/abs/2205.13504.